

UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA
Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Bloque 7 · Supabase y PostgreSQL: persistencia segura · API_TEATRO

ESPACIO ACADÉMICO:
Seguridad en el Desarrollo de Software

PRESENTADO A:
Sergio Arley Puerto Moreno

AUTORES:
Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofía Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

2026

1. El problema: el teatro vivía en la memoria

Al terminar el Laboratorio 10 la API tenía todas sus defensas puestas: la API Key para saber qué aplicación entra, el JWT para saber quién es la persona, el RBAC para decidir qué puede hacer su rol y las guardas de propiedad para decidir sobre qué registros. Pero debajo de todo eso había arreglos de JavaScript. Cerrar la terminal borraba el teatro entero: los eventos, las funciones, las boletas vendidas y hasta los usuarios registrados. Cada arranque volvía a los datos semilla.

En este bloque cambiamos el suelo sin tocar las paredes. Pasamos los datos a PostgreSQL en Supabase y conservamos lo que ya funcionaba:

- **Igual que antes:** Express y sus rutas, los validadores con `matchedData()`, los controladores, el RBAC, las guardas de propiedad, `/api/boletas/mias`, Swagger, Helmet, CORS y el límite de peticiones. Los endpoints responden exactamente lo mismo.
- **Lo que cambió:** los services. Dejaron de leer arreglos y ahora consultan la base.
- **Lo que sigue temporalmente:** bcrypt y nuestro propio JWT. Supabase Auth quedó preparado, pero no se integró (sección 11).

Los tres roles del teatro se mantienen: administrador, taquilla y asistente. Ahora, además, viven en una tabla de la base.

2. El modelo relacional

Diseñamos nueve tablas: `roles`, `usuarios`, `eventos`, `localidades`, `funciones`, `funcion_tarifas`, `funcion_descuentos`, `asistentes` y `boletas`. Todas usan identificadores `BIGINT GENERATED ALWAYS AS IDENTITY`, así que el cliente nunca decide un id.

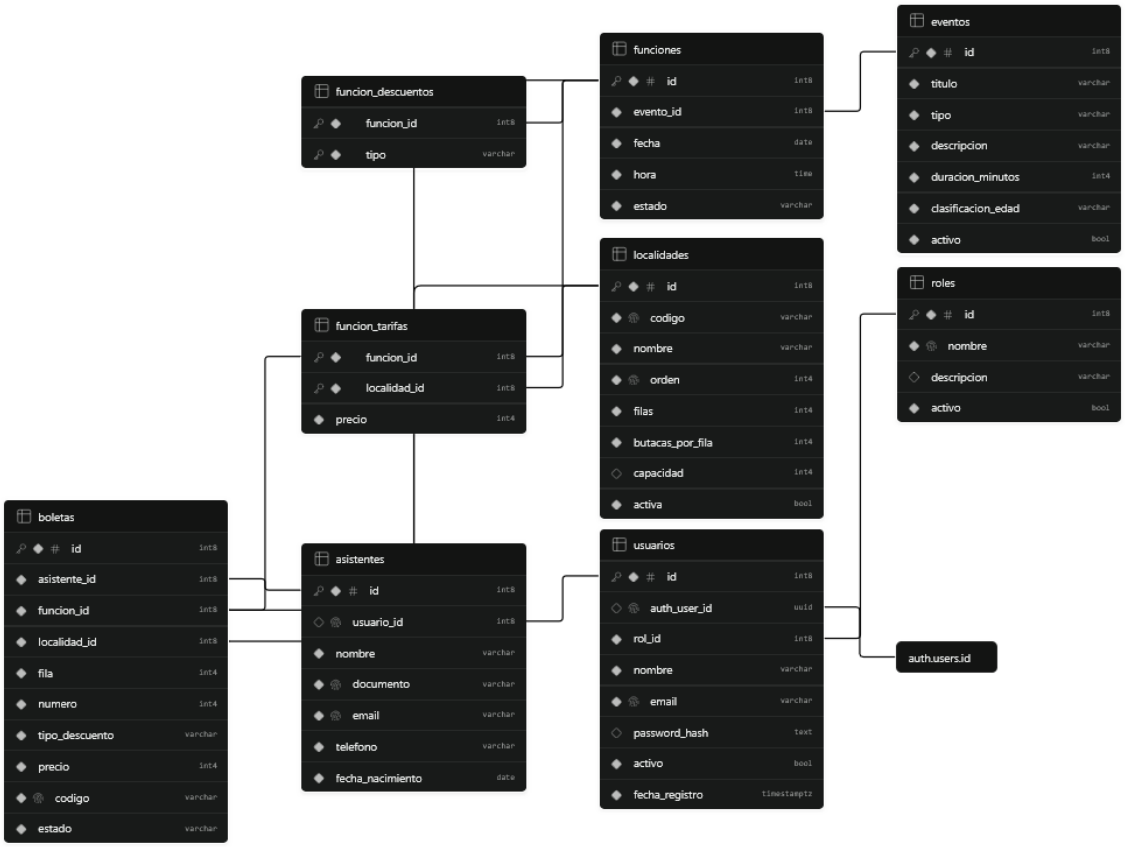


Figura 1. Modelo relacional de API_TEATRO en el visualizador de esquemas de Supabase.

El diagrama lo genera Supabase a partir de las tablas reales. La única relación que sale del esquema `public` es `usuarios.auth_user_id → auth.users.id`, la tabla de Supabase Auth: la columna quedó preparada pero vacía, porque el login sigue siendo el nuestro (sección 11).

Lo que era un arreglo anidado ahora es una tabla

En memoria, una función llevaba sus tarifas y sus descuentos dentro, como arreglos. Eso no se puede guardar así en una base relacional sin perder lo que lo hace útil, así que lo partimos en `funcion_tarifas` y `funcion_descuentos`. La decisión pagó sola: la clave primaria compuesta `(funcion_id, localidad_id)` hace imposible dos tarifas para la misma localidad en una función. Antes esa regla la vigilaba el controlador; ahora no hay camino por el que se pueda violar.

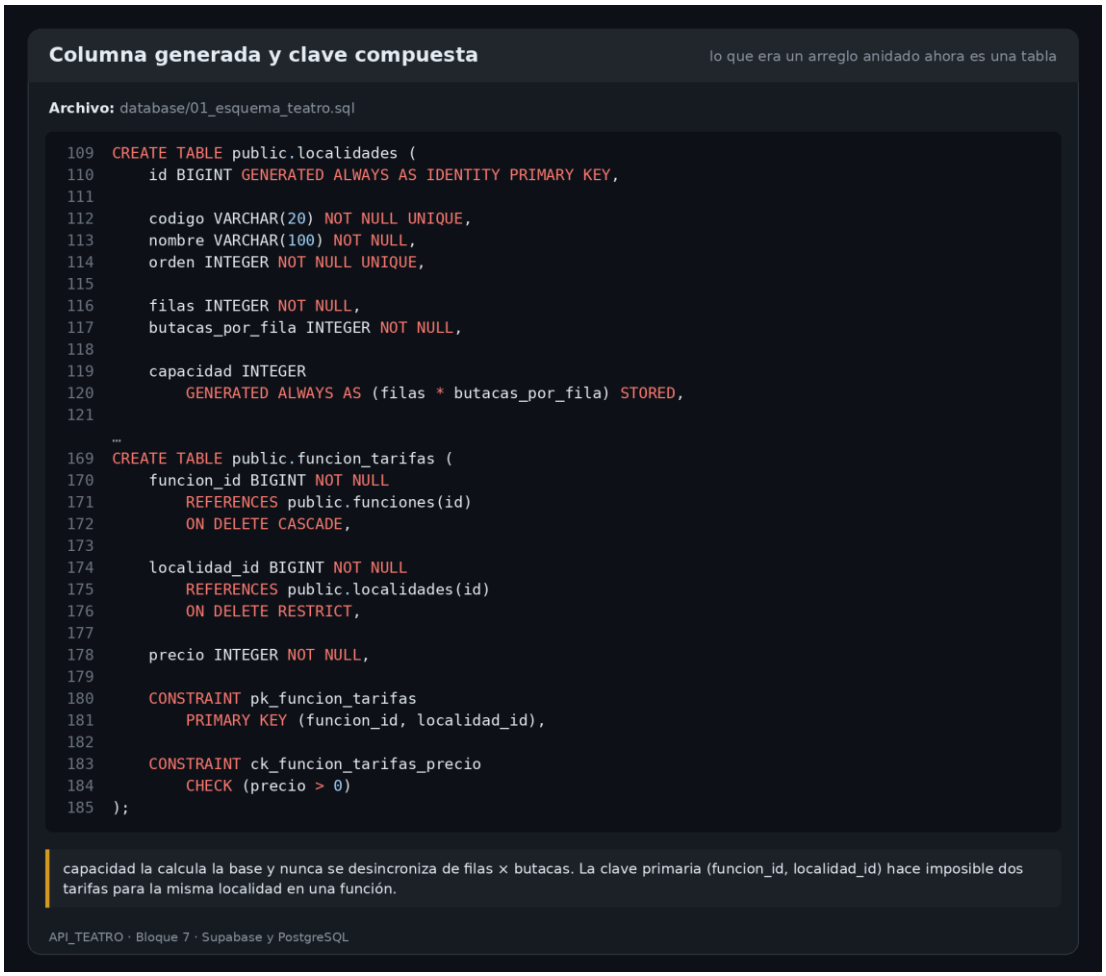


Figura 2. localidades con capacidad como columna generada y funcion_tarifas con clave primaria compuesta.

El id del usuario no es el id del asistente

Son recursos distintos: `usuarios.id` identifica una cuenta que inicia sesión y `asistentes.id` a una persona que compra. Los une `asistentes.usuario_id`, que es `UNIQUE` y admite `NULL`. Si es `NULL`, al asistente lo registró la taquilla y no tiene cuenta; si tiene valor, esa cuenta y solo esa está ligada a ese asistente. El `UNIQUE` es la regla de asociación única del Laboratorio 10, ahora dentro de la base.

3. Reglas del teatro que bajaron a la base

Varias reglas que antes revisaba el código ahora las garantiza PostgreSQL:

Regla	Cómo se garantiza ahora
Sala única: no hay dos funciones a la misma fecha y hora	Índice único parcial `uq_funciones_sala`
Una butaca no se vende dos veces	Índice único parcial `uq_boletas_butaca`
Una boleta solo en una localidad con tarifa en esa función	Clave foránea compuesta `fk_boletas_tarifa`
capacidad = filas × butacas por fila	Columna generada
Tipos, clasificaciones, estados y descuentos válidos	Restricciones `CHECK`
Formato del código de boleta `BOL-AAAA-NNNN`	`CHECK` con expresión regular
Documento, correo, código de localidad y orden únicos	`UNIQUE`



Figura 3. La clave foránea compuesta de boletas y los dos índices únicos parciales.

Por qué los índices son parciales

Los dos índices llevan `WHERE estado <> 'cancelada'`. En el teatro, una función cancelada libera su franja horaria y una boleta cancelada libera su butaca, que se puede volver a vender. El `WHERE` reproduce exactamente lo que hacían `funcionTieneConflicto()` y `butacaOcupada()` en memoria.

Y cierran algo que el código no podía: con un arreglo, dos peticiones al mismo tiempo por la misma butaca podían pasar las dos la comprobación antes de que ninguna escribiera. Un índice único no deja que eso pase.

Lo que no bajó

La coherencia de precios según la cercanía al escenario compara todos los pares de localidades y es más clara en el código. El límite de seis boletas por asistente y función necesitaría un disparador. Y la máquina de estados decide qué cambio se permite, que es distinto de qué valores existen: los valores sí los vigila un `CHECK`.

4. La base en Supabase

Montamos la base con seis scripts SQL en la carpeta `database/`:

- `01_esquema_teatro.sql`: las nueve tablas, sus restricciones e índices.
- `02_roles_seguridad.sql`: los tres roles del teatro, RLS activo en todas las tablas y `REVOKE` para los roles `anon` y `authenticated` de Supabase.
- `03_verificacion.sql`: consultas para comprobar que todo quedó bien.
- `04_datos_semilla.sql`: los datos de `src/data/`, generados desde los mismos archivos.
- `05_rol_aplicacion.sql`: el rol de la aplicación y sus políticas (sección 8).
- `06_vistas.sql`: dos vistas de solo lectura, `v_programacion` y `v_boletas`, que cruzan las tablas para poder revisar los datos en Supabase sin hacer los joins a mano. La API no las usa, y al revisarlas encontramos un problema que corregimos (sección 8).

Con un solo comando, `node scripts/preparar-base.js --reiniciar`, se borra la base, se ejecutan los scripts en orden y se cuentan las filas: 3 roles, 11 eventos, 3 localidades, 16 funciones, 48 tarifas, 29 descuentos, 5 asistentes y 7 boletas, iguales a los de `src/data/`. La tabla `usuarios` arranca vacía a propósito: al administrador lo crea la API en su primer arranque, con la contraseña del `.env`.

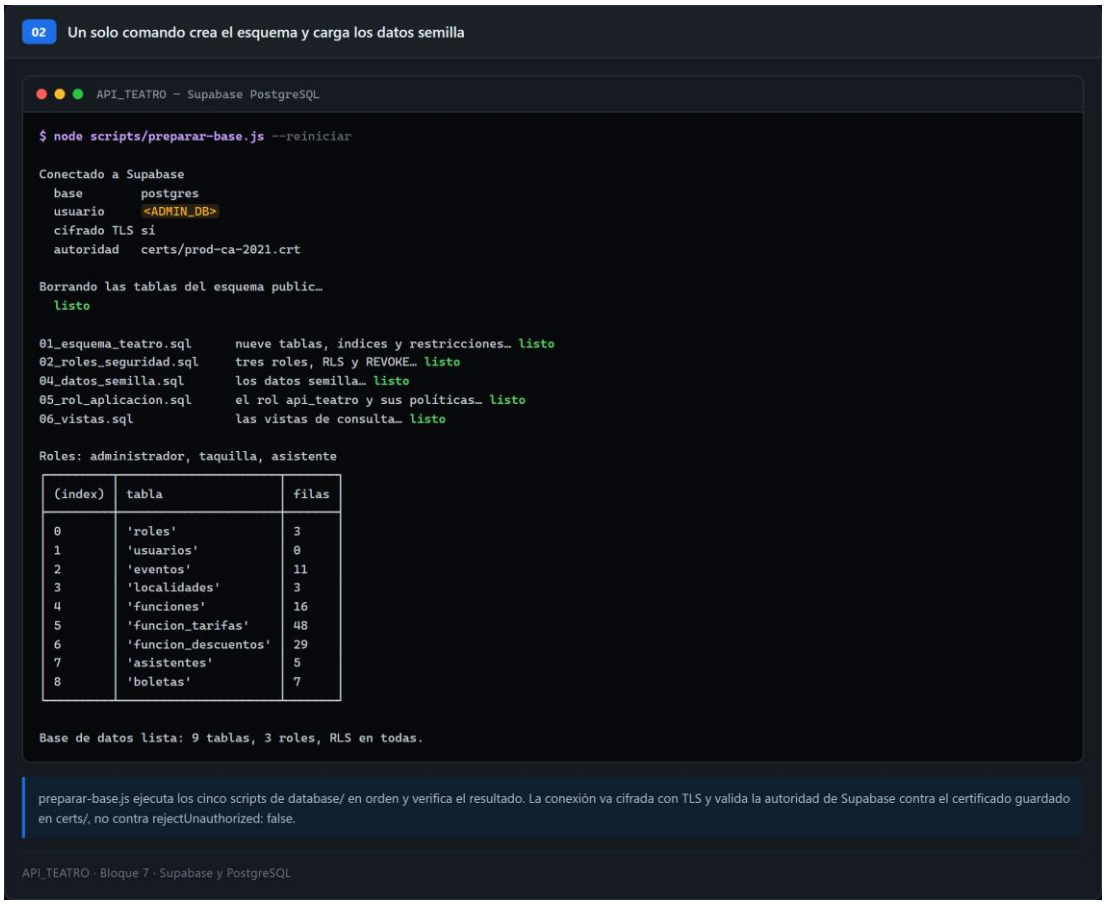


Figura 4. scripts/preparar-base.js --reiniciar: la base se monta de cero sobre TLS y los conteos coinciden con src/data/.

Todas las capturas de este informe salen de ejecuciones reales contra la base de Supabase. En ninguna aparecen la cadena de conexión, el host ni contraseñas: el usuario dueño se muestra como `<ADMIN_DB>`, la contraseña del administrador como `••••••••` y los tokens recortados como `eyJhbGci…`. Las pruebas que intentan escribir datos inválidos van dentro de `BEGIN … ROLLBACK`.

Y comprobamos que ninguna tabla quedó por fuera de Row Level Security:

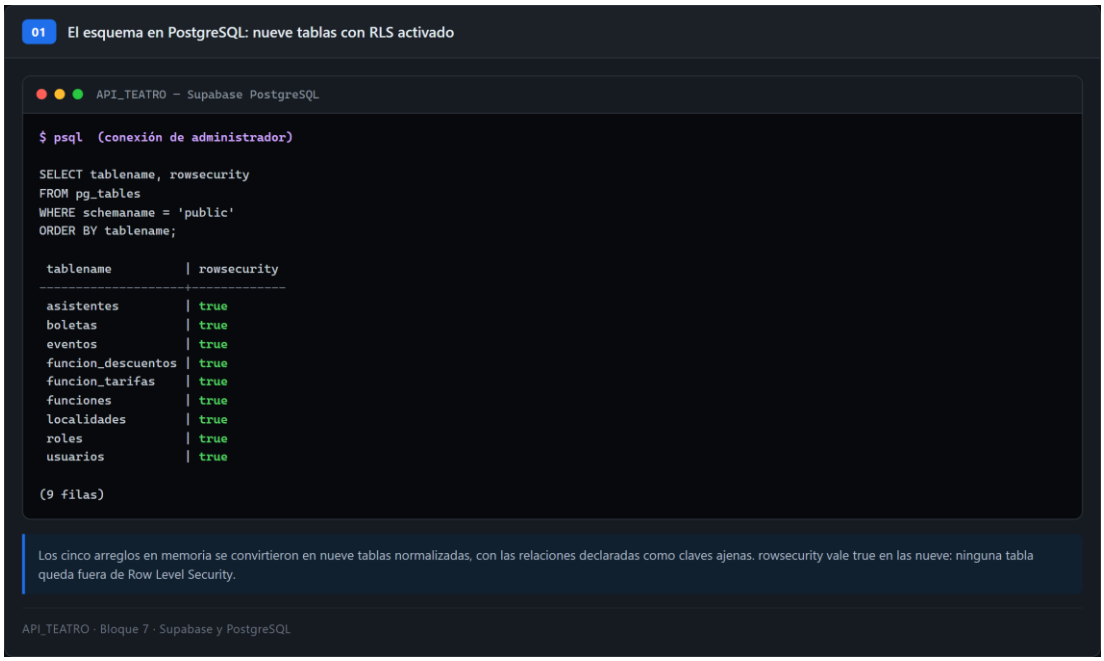


Figura 5. Las nueve tablas del esquema public con rowsecurity = true.

5. La conexión: Express con PostgreSQL

Creamos `src/config/database.js`, que abre un pool de conexiones con TLS. La cadena de conexión vive solo en el `.env`, y si falta el servidor no arranca: preferimos no arrancar a descubrirlo en la primera petición de un usuario.

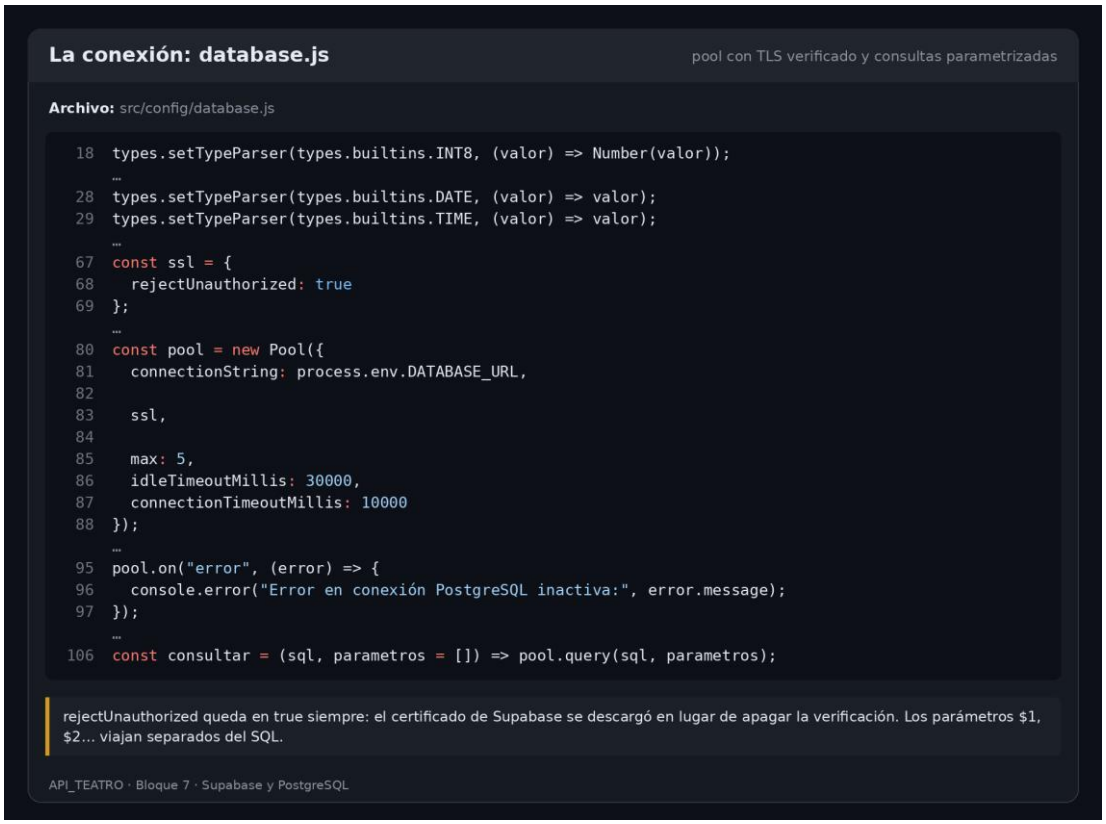


Figura 6. src/config/database.js: tipos, TLS verificado, pool y consultas parametrizadas.

TLS sin apagar la verificación

Durante el montaje apareció el error `SELF_SIGNED_CERT_IN_CHAIN`, porque el endpoint directo de Supabase usa su propia autoridad de certificados. La salida fácil era poner `rejectUnauthorized: false`: la conexión funciona, pero acepta igual de bien a un intermediario que al servidor legítimo. Cifrar sin comprobar con quién se habla no protege de nada. Lo resolvimos descargando el certificado de Supabase, que es público, en `certs/prod-ca-2021.crt`.

Un pool, no una conexión por petición

Abrir una conexión TLS cuesta, y el plan gratuito de Supabase limita cuántas puede haber a la vez. El pool las reutiliza (máximo cinco). También tiene un oyente de errores: una conexión inactiva puede caerse y, sin ese oyente, el error tumbaría el servidor.

Para comprobarlo usamos `scripts/probar-conexion.js`, que se conecta, revisa que la conexión vaya cifrada, con qué rol entra y que pueda leer la tabla `roles`:

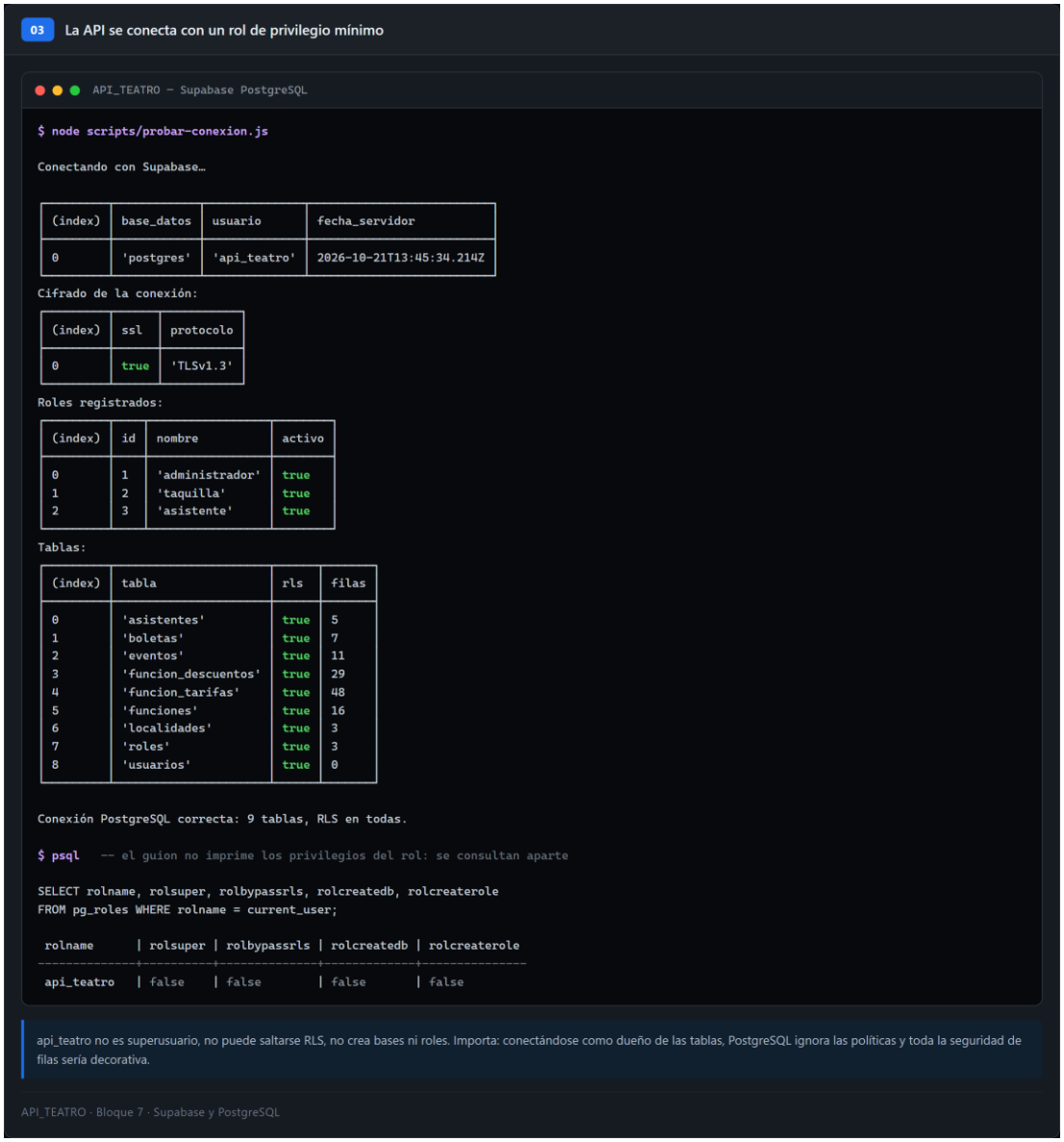


Figura 7. scripts/probar-conexion.js: conexión cifrada con TLSv1.3 como api_teatro, los tres roles leídos de la base y los privilegios del rol, todos en false.

6. Los services pasan a PostgreSQL

Migramos los seis services: localidades, eventos, asistentes, usuarios, funciones y boletas. Cada función conservó su nombre y lo que devuelve; lo único que cambió es que ahora son asíncronas y los controladores las esperan con `await`. De `src/data/` solo sigue vivo `apiKeys.js`, porque las claves de los clientes se comparan por hash en memoria y no son datos del teatro.

Antes: el service leía un arreglo — Laboratorio 10

Archivo: src/services/eventos.service.js (commit 2865661)

```
// ======================================
// Importaciones
// ======================================
const eventos = require("../data/eventos");
…
// ======================================
// Obtener por id
// ======================================
const obtenerEventoPorId = (id) => {
  const evento = eventos.find((e) => e.id === Number(id));
  return evento || null;
};
```

Los datos vivían en src/data/eventos.js y se perdían al cerrar el servidor.

API_TEATRO · Bloque 7 · Supabase y PostgreSQL

Figura 8. Antes: obtenerEventoPorId buscaba en un arreglo.

Ahora: el service consulta PostgreSQL — Bloque 7

Archivo: src/services/eventos.service.js

```
// ======================================
// Importaciones
// ======================================
const { consultar } = require("../config/database");
…
// ======================================
// Obtener por id
// ======================================
const obtenerEventoPorId = async (id) => {
  const { rows } = await consultar(
    `SELECT ${COLUMNAS} FROM public.eventos WHERE id = $1`,
    [Number(id)]
  );

  return aEvento(rows[0]);
};
…
// ======================================
// Actualizar parcial
// Los campos que no llegan conservan su valor
// ======================================
const actualizarEventoParcial = async (id, datos) => {
  const { rows } = await consultar(
    `UPDATE public.eventos
     SET titulo = COALESCE($2, titulo),
         tipo = COALESCE($3, tipo),
         descripcion = COALESCE($4, descripcion),
         duracion_minutos = COALESCE($5, duracion_minutos),
         clasificacion_edad = COALESCE($6, clasificacion_edad)
     WHERE id = $1
     RETURNING ${COLUMNAS}`,
    [
      Number(id),
      datos.titulo ?? null,
      datos.tipo ?? null,
      datos.descripcion ?? null,
      datos.duracionMinutos ?? null,
      datos.clasificacionEdad ?? null
    ]
  );

  return aEvento(rows[0]);
```

Misma función, mismo nombre y misma respuesta; ahora es asíncrona. COALESCE reemplaza al datos.nombre ?? actual.nombre de la versión en memoria.

API_TEATRO · Bloque 7 · Supabase y PostgreSQL

Figura 9. Ahora: la misma función consulta PostgreSQL con parámetros.

Las equivalencias fueron casi siempre las mismas: `find` pasó a ser un `SELECT … WHERE id = $1`, `push` un `INSERT … RETURNING`, `splice` un `DELETE … RETURNING`, y el `datos.nombre ?? actual.nombre` de la actualización parcial se volvió `COALESCE($2, nombre)`.

Los valores van siempre separados del SQL. La librería `pg` sustituye `$1`, `$2`… del lado del servidor, así que ningún dato del usuario se concatena en la consulta. Es la protección contra inyección SQL.

7. Lo que la migración nos obligó a resolver

Ninguno de estos problemas se veía leyendo el código; aparecieron al ejecutar las pruebas.

La librería cambia los tipos

`pg` entrega los `BIGINT` como texto, porque un entero de 64 bits puede superar lo que un número de JavaScript representa sin perder precisión, y convierte `DATE` y `TIME` en objetos `Date`. Sin corregirlo, la API habría empezado a responder `"id": "1"` en vez de `"id": 1`, y fechas como `"1995-03-14T00:00:00.000Z"`. Fijamos tres analizadores de tipo en `database.js` y recortamos los segundos de la hora. La fecha de una función es la del teatro, no un instante universal: arrastrar una zona horaria habría sido incorrecto, no solo feo.

Guardar una función toca tres tablas

Crear o actualizar una función escribe en `funciones`, `funcion_tarifas` y `funcion_descuentos`. Por eso va dentro de una transacción: si algo falla a mitad, se deshace todo y no queda una función sin tarifas.



Figura 10. enTransaccion: BEGIN, COMMIT si todo sale bien y ROLLBACK si algo falla.

Un SELECT junto a un UPDATE no ve el cambio

Al cambiar el estado de una función, la respuesta traía el estado anterior aunque en la base quedaba bien guardado. En PostgreSQL, un `SELECT` que acompaña a un `WITH` que modifica datos trabaja sobre la foto previa a la escritura. Lo corregimos tomando la fila actualizada del propio `RETURNING`.

Un await olvidado no falla

Pasar seis controladores a `async`/`await` fueron cientos de cambios, y un `await` olvidado no da error: deja una promesa donde se esperaba un objeto. Dos casos solo aparecieron al ejecutar: `await servicio.obtener().filter(…)` espera el resultado del `filter` y no el de la consulta, y un `.map(async …)` sin `Promise.all` devuelve promesas. Comparar un campo sobre una promesa da `undefined`, y una regla de negocio habría pasado siempre sin comprobar nada.

8. Privilegio mínimo y Row Level Security

Al principio la API se conectaba a la base con el usuario dueño de las tablas. Al revisarlo vimos dos problemas. Si la cadena de conexión se filtraba, quien la tuviera podía hacer `DROP TABLE`, no solo leer datos. Y, más importante, RLS quedaba desactivado de hecho: el dueño de una tabla no pasa por las políticas, y ese usuario además tenía el permiso `BYPASSRLS`. Escribir políticas sin cambiar esto habría sido de adorno: saldrían en el informe sin filtrar una sola fila.

El rol api_teatro

Con `05_rol_aplicacion.sql` creamos `api_teatro`: sin superusuario, sin `BYPASSRLS`, sin permiso para crear objetos en el esquema y con solo lectura sobre `roles`, que se siembra una vez y no se toca desde la aplicación. Su contraseña la genera `scripts/crear-rol-aplicacion.js` y se copia directo al `.env`; no queda en ningún archivo del repositorio.

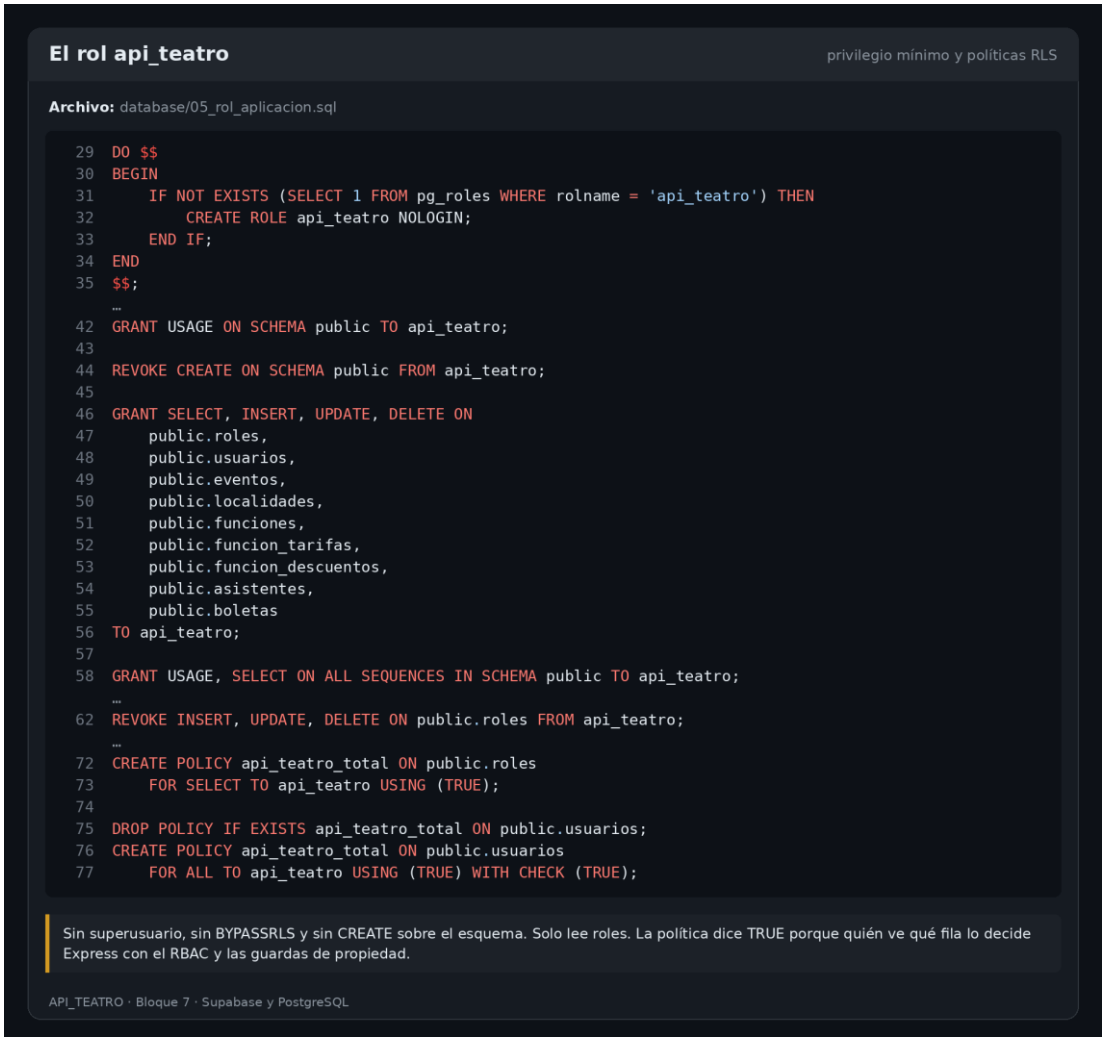


Figura 11. 05_rol_aplicacion.sql: el rol, sus permisos y las políticas.

Ahora hay dos credenciales en el `.env`: `DATABASE_URL` con el rol `api_teatro`, que es la que usa la API, y `DATABASE_URL_ADMIN` con el dueño, que solo se usa para crear o reiniciar la base. Las dos quedan fuera del repositorio y de las capturas.

Comprobamos los límites del rol intentando lo que no debería poder hacer:



Figura 12. Como api_teatro: leer funciona; DROP TABLE, CREATE TABLE, UPDATE roles y desactivar RLS se rechazan con el código 42501.

Que las políticas se aplican de verdad

RLS está activo en las nueve tablas y cada una tiene una política para `api_teatro`. Como RLS niega por defecto a quien no tiene política, `anon`, `authenticated` y cualquier rol futuro ven cero filas aunque alguien les dé un `GRANT` por descuido. Para comprobar que no era de adorno, quitamos la política de `eventos`: el rol pasó de ver 11 filas a ver 0. Al devolverla volvió a ver las 11, y al final comprobamos que las nueve políticas seguían en su sitio. Con el usuario dueño, la misma prueba habría dado 11 en los tres pasos.



Figura 13. Con la política api_teatro ve 11 eventos, sin ella 0 y al restaurarla otra vez 11.

Por qué la política dice TRUE

Porque quién puede ver qué fila lo sigue decidiendo Express: `autorizarRoles` mira el rol del JWT y las guardas de propiedad comprueban que la boleta o el asistente sean de quien pregunta. La API necesita poder leer cualquier fila precisamente para poder responder "403, esa no es tuya". Para que una política filtrara por usuario, PostgreSQL tendría que saber quién es el usuario, y hoy esa identidad vive en nuestro JWT, no en la sesión de la base.

Un hallazgo: las vistas se saltaban RLS

Las dos vistas de consulta las creamos con un comentario que decía que "heredaban las políticas RLS" de las tablas. Al revisarlas vimos que no era cierto. En PostgreSQL, una vista sin la opción `security_invoker` consulta las tablas con los permisos de su dueño, que se salta RLS. Además, Supabase les da permisos por defecto a sus roles `anon`, `authenticated` y `service_role` sobre todo lo que se crea en el esquema `public`.

Lo comprobamos en la base: los tres roles tenían los siete privilegios sobre las dos vistas. `v_boletas` muestra el nombre, el documento y el correo de cada asistente, así que cualquiera con la clave pública de Supabase podía leerlos por su API REST, sin pasar por la nuestra. Era justo lo que el bloque quería proteger.

La corrección fue crear las dos vistas con `WITH (security_invoker = true)`, para que la consulta se evalúe con los permisos de quien la hace, y revocar todo a `PUBLIC`, `anon`, `authenticated` y `service_role`. Después de aplicarla, esos tres roles reciben `permission denied for view` (42501), `api_teatro` sigue leyendo 16 filas de `v_programacion` y 7 de `v_boletas`, y las 356 pruebas siguen pasando.

Es el mismo error que habíamos evitado con el rol `api_teatro`: escribir una protección y no comprobarla. Esta vez la encontramos revisando el código, no ejecutándolo.

9. Cómo lo comprobamos

Las restricciones rechazan lo que deben

Antes de llevar los scripts a Supabase los corrimos contra un PostgreSQL 16 y probamos que las restricciones rechazaran de verdad los datos inválidos: dos funciones a la misma fecha y hora, una butaca repetida, una boleta en una localidad sin tarifa, un código de boleta mal formado, un tipo de evento que no existe, dos asistentes ligados a la misma cuenta y un orden de localidad repetido. Todos fueron rechazados por la restricción

correspondiente, y la misma franja horaria sí se aceptó cuando la función anterior estaba cancelada.

En Supabase lo repetimos como `api_teatro`, dentro de `BEGIN … ROLLBACK` para no dejar rastro: una segunda función el 18 de febrero de 2027 a las 19:00 y una boleta en la fila 2, butaca 7, que ya estaba vendida. La base rechazó las dos con el código 23505, y las boletas siguieron siendo 7.

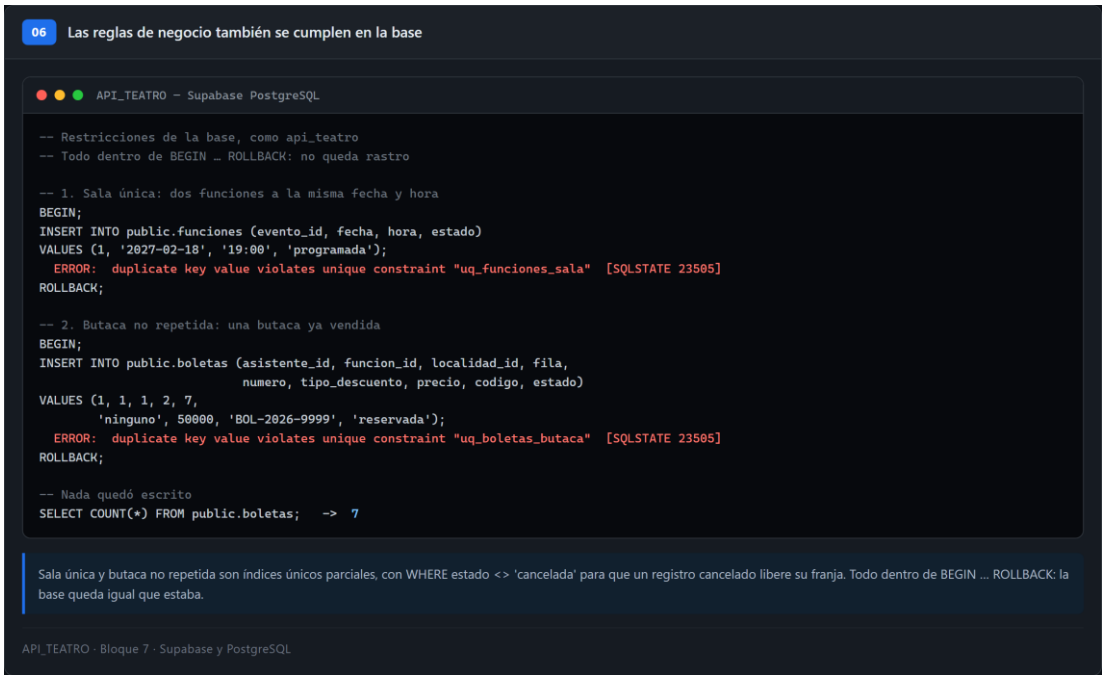


Figura 14. Sala única y butaca no repetida: PostgreSQL rechaza los dos intentos con uq_funciones_sala y uq_boletas_butaca.

Las pruebas automáticas

Las nueve baterías de pruebas de los laboratorios anteriores pasaron sin cambiar ningún resultado esperado: 356 comprobaciones y 0 fallos, ya con el rol `api_teatro`. Hubo un cambio en cómo se corren, y es justamente la prueba de que el bloque funcionó: reiniciar Node ya no devuelve los datos semilla, así que antes de cada batería hay que reiniciar la base con `node scripts/preparar-base.js --reiniciar`.

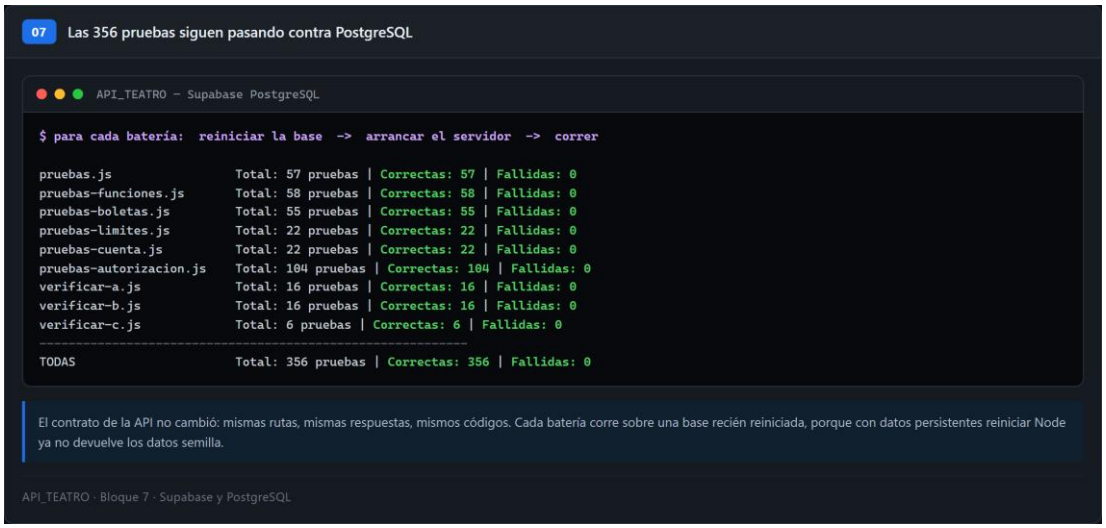


Figura 15. Las nueve baterías contra Supabase, con la base reiniciada antes de cada una: 356 pruebas, 0 fallos.

La prueba que da sentido al bloque

En el primer arranque el servidor creó al administrador. Con él iniciamos sesión y creamos el evento 12, "Prueba de persistencia". Detuvimos el servidor, lo volvimos a levantar y pedimos el evento 12: seguía ahí. Además, el segundo arranque ya no imprimió "Administrador inicial creado", porque la cuenta estaba guardada en la base. Con los arreglos en memoria, las dos cosas se habrían perdido.

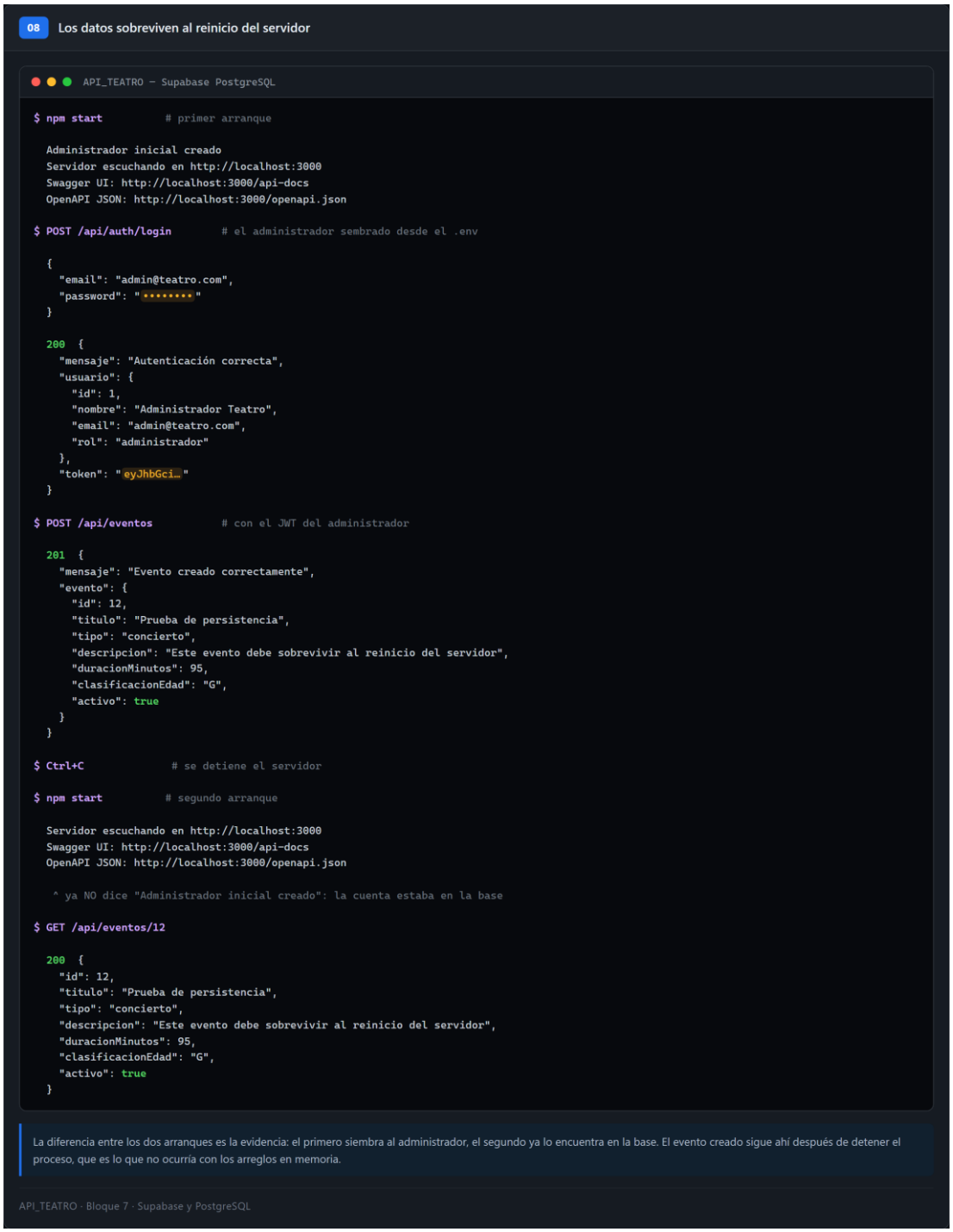


Figura 16. Persistencia: el evento 12 y la cuenta del administrador sobreviven a detener y volver a arrancar el servidor.

También recorrimos la Sala completa como asistente (cartelera, plano de butacas, compra, pago simulado, "Mis boletas" y cancelación) en pantalla grande y en celular. El frontend no necesitó ningún cambio, que era la prueba de que el contrato de la API se respetó.

10. Checkpoint del bloque

Prueba	Esperado	Obtenido	Estado
Conexión TLS con verificación del certificado	Cifrada	Cifrada	OK
Rol de conexión de la API	Sin BYPASSRLS	api_teatro, false	OK
RLS en las nueve tablas	true	9 de 9	OK
Datos semilla contra src/data/	Mismos conteos	122 filas	OK
Dos funciones a la misma fecha y hora	Rechazado	uq_funciones_sala	OK
Misma franja con la anterior cancelada	Permitido	Permitido	OK
Butaca repetida en boleta viva	Rechazado	uq_boletas_butaca	OK
Boleta en localidad sin tarifa	Rechazado	fk_boletas_tarifa	OK
Código de boleta mal formado	Rechazado	ck_boletas_codigo	OK
Dos asistentes en la misma cuenta	Rechazado	UNIQUE usuario_id	OK
api_teatro → DROP TABLE boletas	Rechazado	must be owner	OK
api_teatro → CREATE TABLE	Rechazado	permission denied	OK
api_teatro → UPDATE roles	Rechazado	permission denied	OK
api_teatro → desactivar RLS	Rechazado	must be owner	OK
Quitar la política de eventos	0 filas	11 → 0 → 11	OK
anon, authenticated y service_role → vistas	Rechazado	permission denied	OK
Pruebas automáticas	0 fallos	356 de 356	OK
Evento después de reiniciar el servidor	Sigue ahí	200, evento 12	OK

11. Lo que este bloque no resuelve

- **Supabase Auth.** La columna `usuarios.auth_user_id` existe pero está vacía: las contraseñas siguen en `password_hash` con bcrypt y el login sigue entregando nuestro propio JWT. Integrarlo obliga a cambiar el login y rehacer parte de los laboratorios 9 y 10.
- **Las políticas RLS no filtran por propietario.** Hoy dicen `TRUE`. Para que la base filtrara por usuario habría dos caminos: Supabase Auth con `auth.uid()` en las políticas, o pasar la identidad del JWT a la sesión de PostgreSQL en cada petición.
- **service_role conserva acceso a las nueve tablas.** Es la clave de administración de Supabase y por diseño se salta RLS. La API no la usa y no está en el `.env`; no la

retiramos porque la necesitaría una futura integración con Supabase Auth, pero si se filtrara daría acceso completo.

- **El límite de seis boletas y la coherencia de precios** siguen solo en el código.
- **No hay migraciones versionadas.** Un cambio de esquema hoy se aplica a mano.
- **No hay copias de seguridad propias**: dependemos de las que haga Supabase.

Lo que nos llevamos

Que una restricción en la base y una comprobación en el código no son lo mismo aunque digan lo mismo: el controlador comprueba y después escribe, y entre esas dos cosas puede pasar algo; un índice único no tiene ese hueco. Y que una medida de seguridad que no se comprueba puede ser peor que no tenerla: las políticas RLS sobre un rol que se las salta habrían quedado muy bien en el informe sin proteger nada.

12. Repositorio

https://github.com/JuanK2550/API-TEATRO

La migración quedó en dos commits: la base, la conexión y los seis services en `9db8fce`, y el rol de privilegio mínimo con las políticas RLS en `c205401`. El informe técnico está en `docs/seguridad/informe-bloque7.md` (commit `0a343b0`) las vistas de consulta en `a6fdea3` y su corrección en `60e704a`. La carpeta `database/` tiene su propio README con los pasos para montar la base.